

Шаблоны SQL-запросов

Для упрощения построения сложных запросов с параметрами разработана система шаблонов для SQL-запросов.

Основные цели, для которых создавалась система шаблонов:

- именованные параметры;
- размещение текста запроса в отдельном файле;
- удобство формирования необязательных частей запроса;
- детерминированность разбора шаблона;
- кэширование разобранного шаблона.

Шаблон SQL-запроса представляет собой текстовую строку на особом языке. Шаблон состоит из преамбулы, фрагментов текста запроса, подстановки параметров по названию и управляющих конструкций, которые позволяют перенести часть логики построения запроса в текст шаблона.

Язык шаблонов был разработан на основе анализа существующих систем шаблонов. Синтаксические элементы были выбраны таким образом, чтобы они минимально конфликтовали с языком SQL, но при этом остается возможность их вставки в конечный текст запроса с помощью экранирования. При этом учитывались пожелания о разработке запросов в средах IDE.

Для обеспечения возможности использования любых (в том числе и будущих) диалектов SQL, а также для упрощения разбора текста шаблона было принято решение не разбирать синтаксис SQL. Поэтому язык шаблонов никак не связан с языком SQL, и его разбор ведется без учета синтаксиса SQL. Нужно это учитывать и пользоваться экранированием, если интерпретация каких-то элементов шаблона нежелательна (например, внутри строковых констант SQL).

Поскольку разбор шаблона занимает определенное время, то разобранный шаблон желательно сохранять для повторного использования, если такое использование вероятно в будущем. Разобранный шаблон можно и нужно использовать повторно. Допускается одновременное использование разобранного шаблона из разных нитей, поскольку шаблон после разбора не изменяется.

Основные преимущества использования шаблонов:

- именованные параметры вместо номеров;
- условные конструкции `ifdef`, `ifnotnull`, проверка на равенство с учетом `NULL`;
- списки для `where` и `set`, в которых `where (set)` опускается, если список пуст;
- большие шаблоны в отдельных файлах модуля;
- повторное использование разобранного шаблона.

Предполагается, что при использовании шаблонов снизится вероятность ошибок, ведущих к [SQL-инъекциям](#).

Синтаксис шаблона

Шаблон состоит из фрагментов текста запроса, подстановок параметров и управляющих конструкций. В начале шаблона может идти преамбула, в которой задаются глобальные параметры шаблона.

Преамбула

В начале шаблона может идти необязательная преамбула, состоящая из одной или нескольких строк, каждая из которых начинается с `--meta` и заканчивается маркером конца строки (`\n` или `\r\n`).

Распознаются следующие варианты:

- `--meta prefix=X` задает одиночный символ префикса подстановки параметра.

Подстановка параметров

Формат подстановки параметра: `!параметр`.

Имя параметра может состоять из русских и английских букв, цифр, нижнего подчеркивания, точки, символа `'@'`. В запрос подставляется placeholder (кроме `SqlIdentifier` и `SqlLiteral`, которые записываются прямо в текст запроса с экранированием специальных символов).

Если значение параметра не задано, попытка его подстановки вызывает исключение.

Символ префикса перед именем может быть задан с помощью `--meta prefix=X` в преамбуле шаблона, по умолчанию это знак восклицания.

Для вставки символа префикса в конечный SQL-запрос его нужно удвоить.

Условные конструкции

Варианты ветвления:

- `{% ifdef параметр %}` текст `{% endif %}`
Текст вставляется в запрос, если параметр задан. Здесь и далее текст может включать в себя подстановку параметров и другие управляющие конструкции шаблона.
- `{% ifnotnull параметр %}` текст `{% endif %}`
Текст вставляется в запрос, если параметр задан и не NULL.
- `{% ifndef параметр %}` текст `{% endif %}`
Текст вставляется в запрос, если параметр не задан.
- `{% ifnull параметр %}` текст `{% endif %}`
Текст вставляется в запрос, если параметр задан и является NULL.
- `{% iftrue параметр %}` текст `{% endif %}`
`{% iffalse параметр %}` текст `{% endif %}`
Текст вставляется в запрос, если параметр является булевым значением true (false). Если параметр не задан, является NULL, или не является bool, то вызывается исключение.
- `{% = параметр %}`
Подставляется `!параметр`, если параметр задан и не NULL, иначе подставляется `IS NULL`.
- `{% ifequal параметр "литерал" %}` текст `{% endif %}`
`{% ifnotequal параметр "литерал" %}` текст `{% endif %}`
Текст вставляется в запрос, если параметр равен (ifequal) или не равен (ifnotequal) строковому литералу. Строковый литерал должен быть заключён в двойные кавычки ("), для экранирования символа двойных кавычек в теле литерала следует использовать обратную косую (\). Проверка на равенство производится путём конвертации строкового литерала к типу параметра. Если параметр является NULL, или литерал не может быть приведён к типу параметра, то параметр считается не равным литералу. Если параметр не задан, то вызывается исключение.

Списки для where и set

Список начинается `{% list [параметры] %}`, заканчивается `{% endlist %}`. У команды list есть следующие необязательные параметры:

- `begin` – текст перед началом списка;
- `end` – текст после конца списка;
- `sep` – разделитель элементов списка.

Значения параметров начинаются после знака равенства и заканчиваются пробелом. Значение можно заключить в двойные кавычки, тогда оно заканчивается закрывающей кавычкой. Внутри кавычек можно вставить кавычку или обратную косую с помощью экранирования обратной косой.

Текст в начале и конце списка попадает в конечный текст запроса, только если список не пустой и не состоит из одних пробелов.

Элемент списка начинается `{% item [условие] %}`, заканчивается `{% enditem %}`. Необязательным условием может быть любая конструкция if. При наличии условия элемент вставляется только при выполнении условия. Элемент не вставляется, если он пустой или содержит только пробелы.

Для удобства сделаны специализации списков:

- `{% where all %}`
Эквивалентно `{% list begin="where(" sep=")and(" end=")" %}`.
- `{% where any %}`
Эквивалентно `{% list begin="where(" sep=")or(" end=")" %}`.
- `{% set %}`
Эквивалентно `{% list begin=set sep=, %}`.
- `{% group all %}`
Эквивалентно `{% list begin="(" end=")" sep=")and(" %}`.

- {% group any %}

Эквивалентно {% list begin="(" end=")" sep="")or(" %}%.

Специализированные списки закрываются соответствующим окончанием {% endwhere %}, {% endset %} или {% endgroup %}.

Внутри списков разрешаются только пробелы, элементы item и group, а также подстановочные параметры. При этом, если тип параметра подстановки является массивом, его члены подставляются в запрос в качестве элементов окружающего списка. Параметры иных типов сами подставляются в запрос как элементы списка.

Экранирование

Для вставки в текст запроса символа префикса подстановки параметра он удваивается (например, !! вместо !).

Для вставки в текст запроса строки с {% используется обратная косая:

- {\% преобразуется в {%
- {\} преобразуется в {\

Комментарии

В шаблоне могут содержаться комментарии, которые будут удалены при разборе шаблона и не будут попадать в конечный SQL-запрос.

Комментарий имеет синтаксис: {# комментарий #}

Комментарии могут быть многострочными.

Пробелы

Пробелом считаются символы: пробел, табуляция, перевод строки, возврат каретки. Пробелы после {% и перед %} не обязательны. Несколько пробелов считаются как один.

Вложенность

Все управляющие конструкции, содержащие текст, могут быть вложены. Элементы item должны быть вложены в списковый элемент (list, where, group, set). Списковые элементы должны содержать в себе только пробелы и элементы item, group.

Альтернативное написание

Допускается писать пробел после end, например {% end if %}, {% end item %} и т.д.

Вложенные SQL-шаблоны

Можно компоновать SQL-запрос из нескольких шаблонов с помощью подстановки их в качестве параметра шаблона.

Особенности:

- Экземпляр dbi::TemplateExecutor с назначенными параметрами можно передать в качестве параметра другому dbi::TemplateExecutor.
- При передаче dbi::TemplateExecutor он копируется вместе с назначенными параметрами. Поэтому один и тот же dbi::TemplateExecutor можно передавать несколько раз, меняя его параметры.
- При исполнении шаблона, у которого в параметрах есть другой (вложенный) шаблон, происходит рекурсивное выполнение этого вложенного шаблона. Исполнение вложенного шаблона происходит только при необходимости, при подстановке параметра в текст запроса.
- Подстановка вложенного шаблона выполняется эффективно, его текст записывается сразу в целевой буфер, где формируется текст SQL-запроса.

Пример:

```
1 cte_film = TemplateExecutor(Template(' '
2 SELECT
3     film_id
4     , title
5     , CASE
6     WHEN length < !short THEN
```

```

7         'Short'
8     WHEN length >= !short AND
9         length < !long THEN
10        'Medium'
11    WHEN length >= !long THEN
12        'Long'
13    END length
14 FROM
15     film;
16 '''))
17 cte_film.AddParameter('short', 30)
18 cte_film.AddParameter('long', 90)
19
20 films_query = TemplateExecutor(Template(''
21 WITH cte_film AS (
22     !cte_film
23 )
24 SELECT
25     film_id
26     , title
27     , length
28 FROM
29     cte_film
30 WHERE
31     length = !length AND
32     title LIKE !title_filter
33 ORDER BY
34     title
35 '''))
36
37 films_query.AddParameter('cte_film', cte_film)
38 films_query.AddParameter('title_filter', 'Побер%')
39 films_query.AddParameter('length', 'Long')
40 result = QueryByTemplate(films_query)

```

API C++

Для использования шаблона нужно создать объект [Template](#), инициализировать его строкой или загрузить из файла. При этом происходит синтаксический разбор. При ошибке бросается исключение и объект не меняется.

Для выполнения запроса нужно создать объект [TemplateExecutor](#), указав при этом ссылку на объект Template. Сохраняется ссылка на разобранный шаблон, поэтому при повторной инициализации или загрузке Template существующие объекты TemplateExecutor не меняются и будут исполнять предыдущий шаблон. Одновременно может существовать несколько объектов TemplateExecutor, ссылающихся на один и тот же разобранный шаблон.

Далее нужно задать параметры, значения которых сохраняются внутри TemplateExecutor, с помощью семейства методов AddParam.

После задания параметров можно подготовить текст запроса и привязать параметры к объекту IStatement с помощью метода [SetupQuery](#). При попытке подставить в текст запроса не заданный параметр бросается исключение. SetupQuery возвращает StringView, который действителен до разрушения TemplateExecutor или до следующего вызова SetupQuery.

Подготовленный таким образом текст запроса выполняется с помощью методов семейства Exec на объекте IStatement.

Значения параметров можно менять повторным вызовом AddParam и затем вызывать SetupQuery для повторного исполнения шаблона и формирования нового текста запроса.

Для удобства предусмотрено семейство функций [QueryByTemplate](#).

- [sbis::dbi::Template](#)
- [sbis::dbi::TemplateExecutor](#)

- [sbis::dbi::QueryByTemplate](#)

Рекомендуется кэшировать разобранные шаблоны (объекты `sbis::dbi::Template`). Для этого можно сохранять их в статические члены класса, статические переменные функции, модуля.

API Python

Классы `Template` и `TemplateExecutor` экспортированы в Python (модуль `sbis`).

Функции семейства `QueryByTemplate` дополнены. Порядок аргументов в них произволен, интерпретация определяется типом переданного объекта. Можно передавать `IDatabase`, `IStatement`, `Template`, `TemplateExecutor`, `Record`, `dict`, `collections.abc.Mapping`. Необходимо передать `Template` или `TemplateExecutor`, остальные параметры имеют умолчания.

Значения полей `Record`, `dict` или `collections.abc.Mapping`, а также все именованные параметры передаются в `TemplateExecutor` в качестве параметров шаблона. Именованные параметры имеют приоритет над параметрами, переданными через `Record`, `dict` или `collections.abc.Mapping`.

- [sbis.Template](#)
- [sbis.TemplateExecutor](#)
- [sbis.QueryByTemplate](#)
- [sbis.QueryRecordByTemplate](#)
- [sbis.QueryRowByTemplate](#)
- [sbis.QueryScalarByTemplate](#)

Рекомендуется кэшировать разобранные шаблоны (объекты `sbis.Template`). Для этого можно сохранять их в статические атрибуты класса, функции, модуля.

Управление группами шаблонов

Классы `Template` можно грузить из памяти, либо из строковых констант. Шаблон можно грузить из файла, но путь к файлу должен быть абсолютным. Настройки экранирования выполняются на каждый шаблон отдельно.

Регулярная загрузка шаблона из файла неэффективна, так как шаблон будет каждый раз открывать и анализировать файл. Оптимальный вариант – кэширование разобранных шаблонов (объекты `sbis.Template`).

Для повышения удобства работы используется класс [QueryStorage](#). Он позволяет:

- Один раз задать политику экранирования на хранилище шаблонов;
- Кэшировать один `QueryStorage`;
- Грузить файлы шаблонов не по абсолютным путям, а относительным корням модуля;
- Задать статические параметры;
- Выполнять запросы.

С помощью `QueryStorage` можно организовать хранение шаблонов с некоторыми правилами.

`QueryStorage` можно создать относительно корня модуля или подпути относительно корня модуля.

Пример: в модуле "Зарплата и кадры" создается папка "Templates", в которой хранятся SQL-шаблоны. При создании хранилища шаблонов [указываем](#), что в модуле "Зарплата и кадры" (параметр `module`) в папке "Templates" (параметр `path`) нужно найти SQL-файлы. Затем можно у хранилища [запросить шаблон по имени](#), которое является базовой частью имени SQL-файла и выполнить парсинг шаблона согласно [опциям](#).

Из примера следует, что `QueryStorage` – это средство организации каталога шаблонов, благодаря которому можно группе шаблонов указать расположение, схему именования и одни опции. Далее шаблоны можно просто получить по имени, так как они кэшируются в `QueryStorage`. При запросе по имени, которое равно базовой части имени SQL-файла, кэшированный шаблон лежит в `QueryStorage`.

Основная задача `QueryStorage` – упрощение кэширования с помощью задания общих настроек на некоторую группу шаблонов общих политик за раз.

Некоторые параметры шаблонов, общие для всех, можно задавать на уровне `QueryStorage`, т.е. возможно, что некоторый параметр SQL-файлов определяется конфигурацией сервиса.

Если группе связанных шаблонов необходимо задать значение одного и того же параметра (например, `!short`), то этот параметр задается при создании и настройке хранилища. При дальнейшей настройке шаблона параметр указывать не придется за счет того, что шаблон получен из хранилища и параметр уже заполнен. Значение параметра будет равно указанному в `QueryStorage`.

Для C++ документация по `QueryStorage` описана [здесь](#).

Примеры

Пример 1. C++

```
1 static dbi::Template const tmp1( L"insert into !table values(!id,
  !name)"_sv );
2 dbi::TemplateExecutor tmp1_exec( tmp1 );
3 tmp1_exec.AddParam( L"table"_sv, dbi::SqlIdentifier( L"таблица"_sv ) );
4 tmp1_exec.AddParam( L"id"_sv, 1 );
5 tmp1_exec.AddParam( L"name"_sv, L"название" );
6 auto st = db.CreateStatement();
7 st->Exec( tmp1_exec.SetupQuery( *st ) );
```

Выполнение будет эквивалентно такому запросу:

```
1 auto st = db.CreateStatement();
2 st.SetParam( 1, 1 );
3 st.SetParam( 2, L"название" );
4 st.Exec( L"insert into \"таблица\" values($1, $2)"_sv );
```

Пример 2. Python

```
1 def func():
2     if not hasattr(func, 'tpl'):
3         func.tpl = Template("""
4             SELECT * FROM "Клиент"
5             LEFT JOIN
6                 "Договор"
7                 ON "@Клиент" = "Договор"."КлиентСБИС"
8                 AND "@Договор" {% = contract %}
9             WHERE
10                 "@Клиент" = !account
11             """)
12     return QueryByTemplate(func.tpl, contract=None, account=1234)
```

Пример 3. Python

```
1 class KktFetcher:
2     tpl = Template("""
3         SELECT * FROM "KKT" kkt
4         {% where all %}
5             {% item ifnotnull vision_partners %}
6                 kkt."Partner" = ANY(!vision_partners)
7             OR
8                 kkt."Partner" IS NULL
9             {% enditem %}
10            {% item ifnotnull vision_regions %}
11                kkt."Region" = ANY(!vision_regions)
12            OR
13                kkt."Partner" IS NOT NULL
14            {% enditem %}
15            {% item ifnotnull region %}
16                ( -- Регион не указан
17                  !region = -1::int
```

```

18         AND kkt."Region" IS NULL
19     )
20     OR
21     kkt."Region" = !region
22 {% enditem %}
23 {% item ifnotnull partner %}
24     ( -- Незакрепленные
25         !partner = -1::int
26         AND kkt."Partner" IS NULL
27     )
28     OR
29     kkt."Partner" = !partner
30 {% enditem %}
31 {% item ifnotnull manager %}
32     kkt."Manager" = !manager
33 {% enditem %}
34 {% item ifdef with_out_manager %}
35     -- Без ответственного сотрудника
36     !with_out_manager IS NOT TRUE
37     OR
38     kkt."Manager" IS NULL
39 {% enditem %}
40 {% end where %}
41 """)
42
43 def query(self):
44     return QueryByTemplate(self.tmpl, {
45         'vision_partners': [1, 2, 3],
46         'vision_regions': None,
47         'region': -1,
48         'partner': 123,
49         'manager': 55,
50         'with_out_manager': False,
51     })

```

Пример 4. Python

```

1  tmpl = CreateTemplateFromFile('Module', 'sql/query.tmpl')
2  def query_data():
3      return QueryByTemplate(tmpl, record, param=value)

```

Пример 5. Python

```

1  storage = sbis.QueryStorage('Registers',
2  'registers/usn_registers/usn/sql/transcription')
3  return storage.Query('envd_tree', **_params)

```

Пример 6. C++

```

1  {
2      static dbi::QueryStorage qstor( MODULE_NAME, TEMPLATE_DIR_NAME );
3
4      auto exec = qstor.GetTemplateExecutor( FAST_MENU_TEMPLATE_NAME );
5

```

```
6     exec.AddParam( L"row_count"_sv, rows );
7     exec.AddParam( L"column_count"_sv, cols );
8     exec.AddParam( L"limit"_sv, limit );
9     exec.AddParam( L"offset"_sv, offset );
10
11     auto& db = catalog::GetDatabase();
12     result = dbi::QueryByTemplate( db, exec );
13     result.ChangeColumnFormat( L"IsFolder", *CreateFieldFormat(
14         L"IsFolder", ftBOOLEAN ) );
15 }
```